BLE PPG Streaming Interface Specification

For mobile app discovery, subscription, parsing, plotting, and packet-loss handling

1. Purpose

This document defines the current BLE interface exposed by the ESP32-based PPG sensor board. The mobile app should use this interface to discover the device, connect, enable notifications on the data characteristic, and parse incoming PPG packets for live plotting and downstream machine-learning processing.

2. Discovery and connection

Advertised device name	Sensor-board
BLE role	Peripheral / GATT server
Primary service UUID	a1f6f5ff-bf33-4d42-9a21-e45dad587e23
Data characteristic UUID	a1f6f5ff-bf33-4d42-9a21-e45dad587e24
Characteristic properties	Notify, Read

Mobile app flow:

- Scan for a peripheral advertising the local name PPG-Stream.
- Connect to the device.
- Discover the primary service a1f6f5ff-bf33-4d42-9a21-e45dad587e23.
- Discover the data characteristic a1f6f5ff-bf33-4d42-9a21-e45dad587e24 under that service.
- Enable notifications on the data characteristic.
- When notifications are enabled, incoming packets should be parsed using the binary format in Section 3.

3. Notification packet format

Each BLE notification contains exactly one complete PPG packet. Packet size is variable. The receiver must not assume a fixed byte length; it must parse the header and then decode the number of samples indicated by the sample_count field.

Offset	Size	Field	Description
0	4	packet_seq	Unsigned 32-bit packet sequence number, little-endian.
4	2	sample_count	Unsigned 16-bit count of samples present in this packet, little-endian.
6	4 × N	samples	N unsigned 32-bit PPG samples, little-endian. N = sample_count.

Current firmware assumptions:

- Maximum samples per packet: 50
- Sample representation: uint32
- Valid sensor bits: lower 18 bits currently contain the PPG value
- Endianness: little-endian for all fields and samples
- Payload is variable because the firmware batches samples based on buffer availability and send threshold

4. Example packet layout

Example: packet_seq = 25, sample_count = 3, samples = [1000, 1010, 1020]

```
19 00 00 00    packet_seq = 25
03 00          sample_count = 3
E8 03 00 00    sample[0] = 1000
F2 03 00 00    sample[1] = 1010
FC 03 00 00    sample[2] = 1020
```

5. Parsing requirements on the mobile side

- Treat each notification as one self-contained packet.
- Read the first 4 bytes as packet_seq (uint32, little-endian).
- Read the next 2 bytes as sample_count (uint16, little-endian).
- Validate that total notification length equals 6 + (4 × sample_count) bytes.
- Decode exactly sample_count samples, each as uint32 little-endian.
- For plotting or ML preprocessing, use the numeric sample values directly from the uint32 array.
- Only the lower 18 bits are currently meaningful; upper bits may be zero.

6. Packet loss handling

BLE notifications are used for throughput, so the transport is effectively fire-and-forget at the application level. The app should detect missing packets by tracking packet_seq.

- Expected behavior: packet_seq increments by 1 for each packet.
- If the app receives seq values 100, 101, 104, then packets 102 and 103 were missed.
- Missed packets imply missed samples. The exact number of missed samples depends on the sample_count values of the lost packets, which are not recoverable unless a retransmission scheme is added later.
- The app should log sequence gaps for debugging and optionally mark dropped regions in plots or ML windows.
- Current firmware does not provide retransmission or app-level acknowledgment.

7. Current behavior and practical notes

- The ESP streams as notifications only after a central has connected and enabled notifications on the data characteristic.
- The firmware currently batches samples from a shared circular buffer before transmission.
- The firmware is currently tuned to reduce packet overhead by waiting for larger batches; packet sizes are therefore variable.
- Some packet loss may still occur during early tuning. The packet_seq field is the intended mechanism for app-side detection.
- The app should be written to tolerate variable packet sizes and occasional sequence gaps.

8. Suggested mobile-side validation checklist

- Confirm the device can be discovered by name: Sensor-Board.
- Confirm notifications can be enabled on characteristic a1f6f5ff-bf33-4d42-9a21-e45dad587e24.
- Confirm packet lengths always satisfy 6 + 4 × sample_count bytes.
- Log packet_seq and detect gaps.
- Store decoded samples to disk during development to validate waveform shape.
- Plot live samples and visually confirm reasonable PPG structure.
- Record packet-loss statistics during extended streaming tests.

9. Future protocol extensions (not required for v1)

- Add first_sample_index or timestamp to each packet for more precise gap reconstruction.
- Add a protocol version byte if the packet format evolves.
- Add a control characteristic for app-to-device commands such as start/stop streaming or configuration.
- Add app-level acknowledgments only if later testing shows they are needed.